↳ In last chapter, we put entire address of each process in memory.

This led to internal fragmentation i.e chunks of free space inside allocate memory.

↳ The space b/w heap & stack is NOT being used by the program but it is still taking up physical space when we relocate the entire address space in physical memory.

Thus, using a base & bounds registers is wasteful.

It also makes it quite hard to run a program, when an entire address doesn't fit into memory.

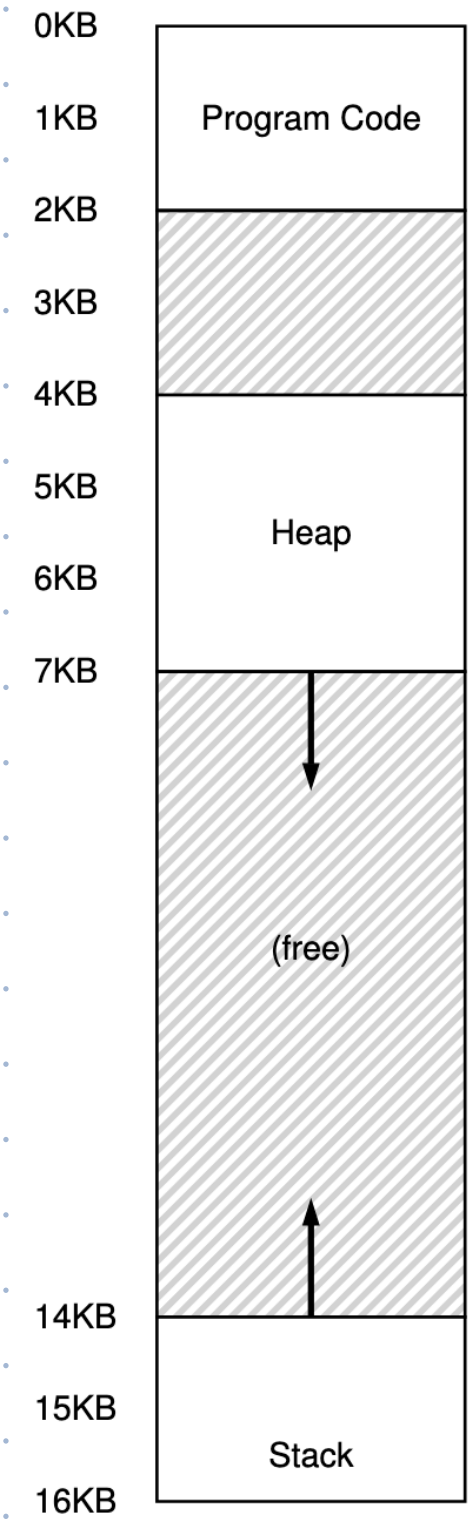


Figure 1

> THE CRUX: HOW TO SUPPORT A LARGE ADDRESS SPACE
> How do we support a large address space with (potentially) a lot of free space between the stack and the heap? Note that in our examples, with tiny (pretend) address spaces, the waste doesn't seem too bad. Imagine, however, a 32-bit address space (4 GB in size); a typical program will only use megabytes of memory, but still would demand that the entire address space be resident in memory.

Segmentation: Generalized Base/Bounds

↳ Instead of having one base & bound pair in our MMU, we have a base & bound pair per logical segment

A segment is just a contagious portion of address space of a particular length.

↳ Thus, segmentation allows the OS to place each segment in different parts of physical memory, and thus avoid filling physical memory with unused virtual space.

For example,

- The address space in figure 1 is placed in physical as in figure 2.
- With a base & bound pair per segment, we can place each segment independently in physical memory.
- Only used memory is allocated space in figure 2.

Thus, large address spaces with large amounts of unused address space, called sparse address spaces, can be accommodated.

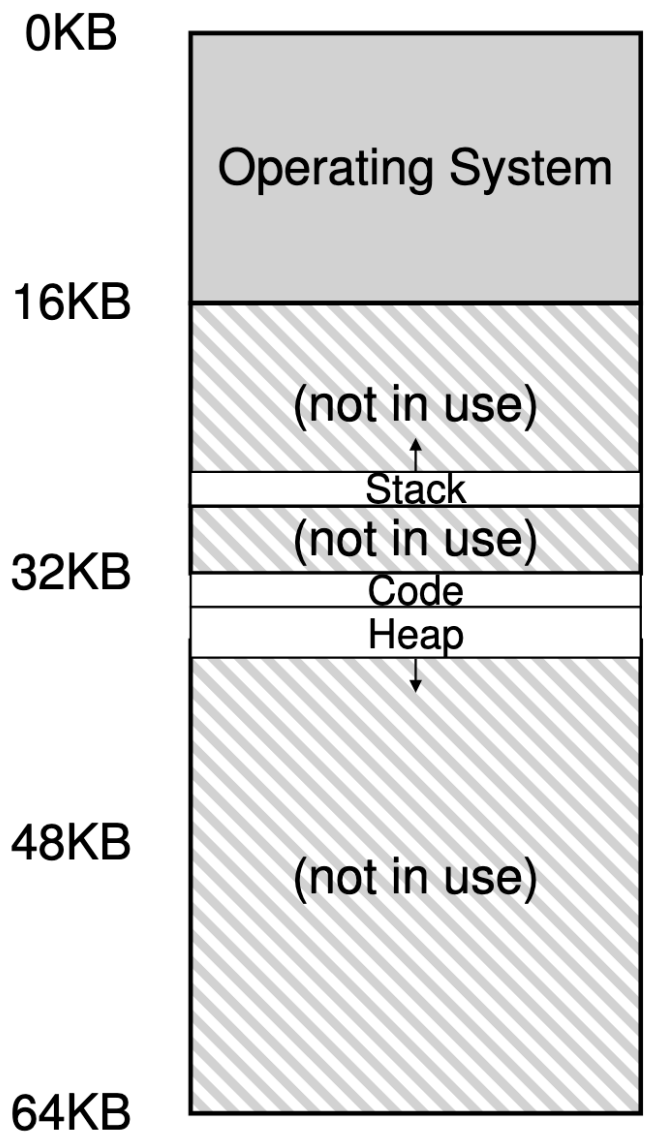


figure 2

Thus, our MMU's base & bound (size) register for each segment would look like:

Segment	Base	Size (bounds)
Code	32K	2K
Heap	34K	3K
Stack	28K	2K

Example translation,

- Assume a reference is made to a virtual address 100 (which is in code segment figure 1)

The hardware will add base value to this offset to get the actual address as:

$$32kB + 100 = 32868$$

It will check that the address is within bounds i.e $100 < 2kB$, find that it is, and issue a reference to physical address 32868.

- Assume virtual address 4200 (which is in heap segment), we get physical address as:

$$34kB + 4200 = 39016$$

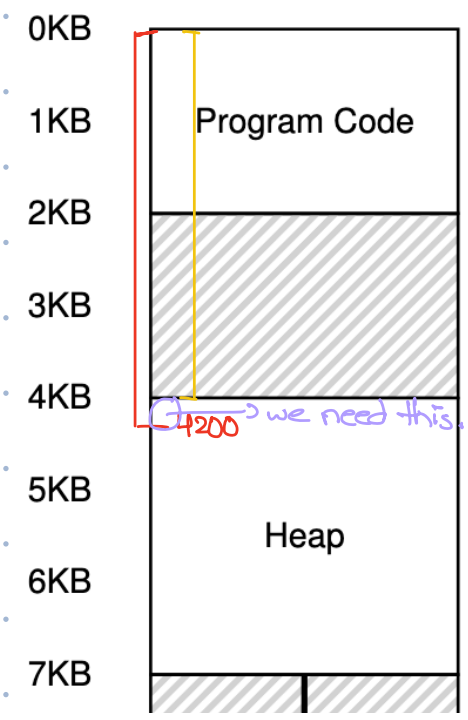
but this is not the correct physical address.

We need to extract the offset into heap. As the heap starts at address 4kB (4096) & we have virtual address 4200, then correct offset is

$$4200 - 4096 = 104$$

We add this to the base register to get correct physical address:

$$34kB + 104 = 34920.$$



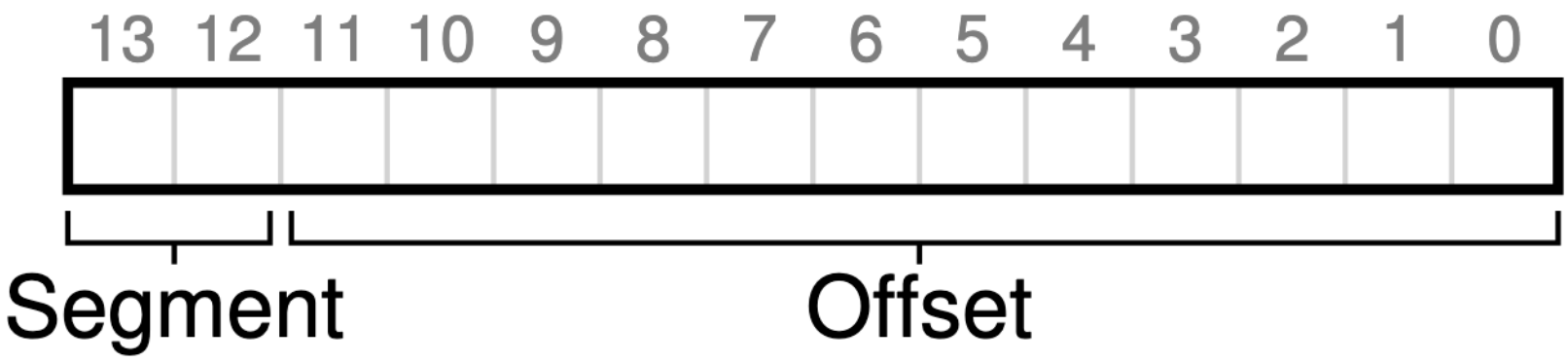
If we reference beyond 7kB, then the hardware detects out of bounds, traps into the OS & this is where segmentation fault comes from.

Which segment are we refering to?

- How does hardware know the offset into a segment, and to which segment it refers to?

Explicit Approach:

Divide the 14 bit virtual address as:



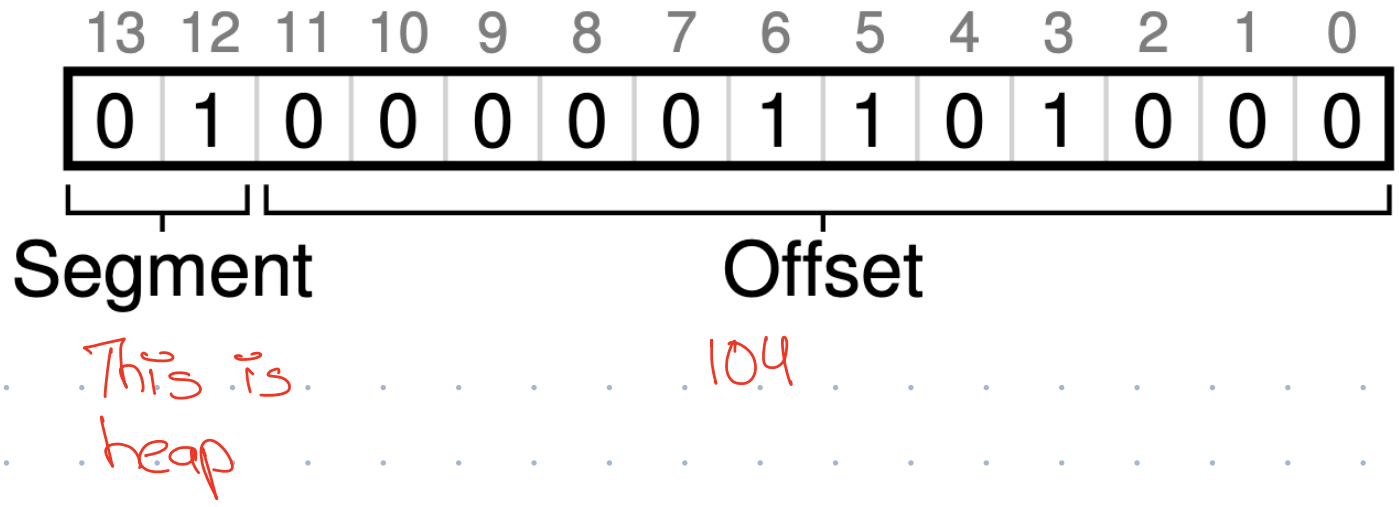
If (13 & 12) bits are 00, the hardware knows its the code segment & uses code segment's base & bound registers for address translation.

If they are 01, the hardware knows its the

heap segment.

Example,

Our virtual address of 4200 looks like



This makes bound checking easier as we check if offset < bounds for legal address.

The hardware does something like below:

```
1  // get top 2 bits of 14-bit VA
2  Segment = (VirtualAddress & SEG_MASK) >> SEG_SHIFT
3  // now get offset
4  Offset  = VirtualAddress & OFFSET_MASK
5  if (Offset >= Bounds[Segment])
6      RaiseException(PROTECTION_FAULT)
7  else
8      PhysAddr = Base[Segment] + Offset
9      Register = AccessMemory(PhysAddr)
```

(12)

↳ This gives 11 000000000000 which is a big number. So, we right shift by 12 to get: 000000000011

Issues with this approach:

1. With three segments, one segment of the address is unused i.e 10

   So, some systems put code & heap in same segments & use only one bit.

2. We limit the size of each segment to a maximum size which is 4kB.

   If the program wishes to grow the segment > 4kB, it is out of luck.

Implicit Approach:

Hardware determines the segment by noticing how the address was formed.

Example,

If the address was from PC, then it is in the code segment.

If the address is based on stack or base pointer, it is in stack segment

Any other address must be in heap.

What about stack?

↳ The stack grows backwards.

↳ In physical address, it starts at 28kB & grows back to 26kB, corresponding to virtual addresses 16kB to 14kB.

So, translation must be done differently.

↳ The hardware along with base & bound values needs to know which way the segment grows.

Segment	Base	Size (max 4K)	Grows Positive?
$Code_{00}$	32K	2K	1
$Heap_{01}$	34K	3K	1
$Stack_{11}$	28K	2K	0

Example,

Assume virtual address 15kB, which should map to physical address 27kB.

15360: 11 1100 0000 0000: 0x3C00

3kB

From top two bits, hardware knows this is stack.

To obtain the correct negative offset, we subtract the maximum segment size (4kB) from segment offset (3kB)

$$3kB - 4kB = -1kB$$

We add this offset to base value to get correct physical address:

$$28kB + (-1kB) = 27kB$$

The bounds check is: abs(offset) < segment size

Support for sharing:

- To save, sometimes it is useful to share certain memory segments address spaces.

  In particular, code sharing is common & still in use in systems today.

- Hardware needs to provide little extra support in the form of protection bits.

Basic support adds few bits per segment indicating whether or not a program can read or write a segment, execute code that lies within the segment.

By setting a code segment to read-only, the same code can be shared across multiple processes

Segment	Base	Size (max 4K)	Grows Positive?	Protection
$Code_{00}$	32K	2K	1	Read-Execute
$Heap_{01}$	34K	3K	1	Read-Write
$Stack_{11}$	28K	2K	0	Read-Write

With protection bits, the hardware algorithm described earlier would also have to change. In addition to checking whether a virtual address is within bounds, the hardware also has to check whether a particular access is permissible. If a user process tries to write to a read-only segment, or execute from a non-executable segment, the hardware should raise an exception, and thus let the OS deal with the offending process.

Fine-grained Vs Coarse grained segmentation

Large & few segments ⟶ Coarse grained Segmentation

Small & alot of segments ⟶ fine grained segmentation

- Requires more hardware support, with a segment

table.

- The thinking at the time was that OS could better learn which segments are in use & which are not and better utilize memory.

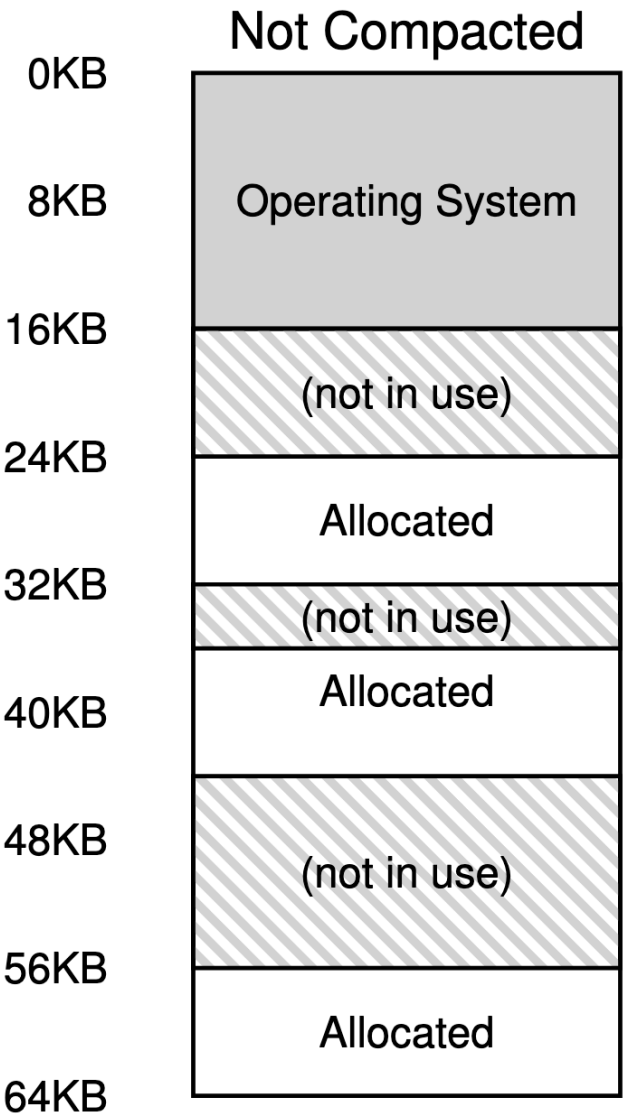
OS support

- To context switch, segment registers must be stored.
- In case heap grows, OS must update the bounds register.
- When a new address space is created, OS has to be able to find space in physical memory for it's segments.

Previously, we assumed that each address space was the same size, thus physical could be though of bunch of slots where processes would fit in.

Now, we have no.of segments per process each of different size

- The general problem that arises is that physical memory quickly becomes full of holes. We call this external fragmentation.



- In right memory, if a process wishes to allocate 20kB then its request will be reject as there is 24kB but in non-contagious chunks.

Solutions:

1. Compact the physical memory be re-arranging the existing segments.

   Example,
   OS could

1. stop running processes

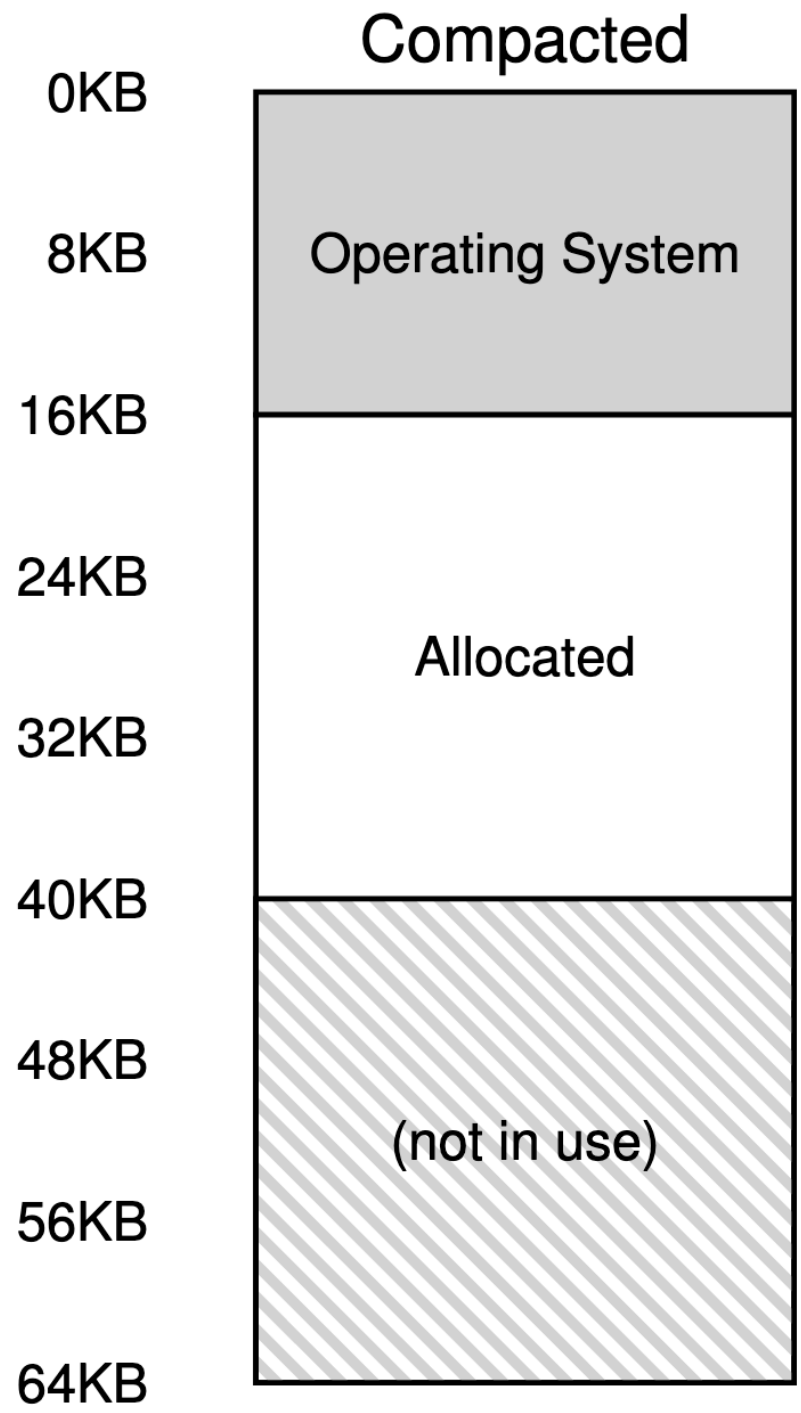
2. copy their data to one contagious region of memory

3. change their segment register values to point to the new physical locations.

In this way, we have large extent of free contagious memory.

However, compaction is expensive, as copying segments is memory intensive.

This also makes requests to grow segments hard to serve.



2. Free-list management

↳ keeps track of large extent of free memory.